1. What is a transpiler? Can you name a popular programming language that uses a transpiler?

A transpiler is a program that converts source code written in one programming language or version into another language or version at a similar level of abstraction. Unlike a traditional compiler that usually converts high-level code into machine code, a transpiler often converts one high-level language into another.For example, TypeScript is transpiled into JavaScript before it can run in a browser or JavaScript runtime.

2. What are the executable container formats used in Windows 11 and macOS?

In Windows 11, common executable formats include:

- .exe - executable application
- .dll - dynamic-link library
- .msi - Windows Installer package

Windows primarily uses the Portable Executable format for executable and library files.

3. Do you know of any notation used to describe the syntax of programming languages? Try to express an email address in one of these notations.

Yes. Backus-Naur Form and Extended Backus-Naur Form are commonly used to describe the syntax of programming languages.

A simplified EBNF-style representation of an email address can be written as:

email = local-part, "@", domain ;

local-part = letter, { letter | digit | "." | "_" } ;

domain = letter, { letter | digit | "." | "-" } ;

letter = "A" | "B" | so on | "Z" | "a" | "b" | so on| "z" ;

digit = "0" | "1" |so on | "9" ;

For example, an address such as student@example.com follows this general structure.

This is a simplified representation and does not cover every rule allowed by the actual email standard.

4. Do you think a program without functions will be faster than a program for the same logic implemented with functions? Why?

Not necessarily. The presence of functions does not automatically make a program slower.

When a function is called, there can be a small overhead for creating a stack frame, passing arguments, and returning a result. However, modern compilers and interpreters can optimize function calls, including techniques such as inlining.Functions also make programs easier to organize, reuse, test, and maintain. Therefore, removing functions only for performance reasons is generally not a good programming practice unless profiling shows that function-call overhead is actually important.

5. Can errors in call stack or stack frame lead to program errors like wrong functionality or program crashes?

Yes. Errors involving the call stack or stack frames can cause incorrect program behavior or crashes.For example, if a program uses more stack memory than available, it can produce a stack overflow. Incorrect memory handling can also corrupt stack data, which may cause variables, return addresses, or other control information to be interpreted incorrectly.Therefore, problems involving stack memory can result in unexpected output, incorrect execution, or program termination.

6. Do you think security-related vulnerabilities could be associated with call stacks? Can you think of an example?

Yes. The call stack can be involved in security vulnerabilities.

One well-known example is a stack-based buffer overflow. If a program writes more data into a stack buffer than the allocated space, it may overwrite nearby stack information. In vulnerable software, this can potentially affect control-flow information such as a return address.For this reason, modern systems use protections such as stack canaries,

Address Space Layout Randomization, and non-executable memory protections to make stack-based attacks more difficult.

7. How will the setup of the call stack work for a recursive function call?

For every recursive function call, the program creates a new stack frame.

The frame generally contains information such as:

- Function parameters
- Local variables
- Return information
- Other execution-related data

For example, if factorial(3) calls factorial(2), which calls factorial(1), the stack grows like this:

factorial(1)

factorial(2)

factorial(3)

main()

When the base case is reached, the functions return one by one and their stack frames are removed in the reverse order.

8. Can a call stack grow indefinitely? What happens if a function is called recursively too many times?

A call stack normally has a limited amount of memory, so it cannot grow indefinitely.

If a recursive function keeps calling itself without reaching a proper base case, additional stack frames continue to be created. Eventually, the available stack memory may be exhausted.This condition is called a stack overflow and can cause the program to terminate or raise a stack overflow error.

9. What does it mean to say that functions or expressions can have "side effects"?

A side effect occurs when evaluating a function or expression changes something outside its immediate result.

For example: count = count + 1

The expression does not only produce a value; it also changes the value stored in count.

Another example is a function that writes data to a file, modifies a global variable, updates a database, or prints something on the screen.A function that only calculates and returns a result without changing external state is generally considered to have fewer or no observable side effects.

10. How can one stop the execution of a loop before it is completed? Illustrate the difference between break and continue.

i. The break statement immediately terminates the loop.
ii. The continue statement does not terminate the loop. Instead, it skips the remaining statements of the current iteration and moves to the next iteration.

Example using break:

```
for i = 1 to 10
   if i = = 5
      break
   print i
```

Output:

1 2 3 4

Here, the loop completely stops when i becomes 5.

Example using continue:

```
for i = 1 to 5
   if i == 3
      continue
   print i
```

Output:

1 2 4 5

Here, only the iteration for i = 3 is skipped.

11. Write the pseudo code to find the sum of the cubes of the first 9 positive integers.

START

sum ← 0

FOR i ← 1 TO 9

    sum ← sum + (i * i * i)

END FOR

PRINT sum

END

The calculation is:

$1^3 + 2^3 + 3^3 + ... + 9^3 = 2025$

12. Write the pseudo code to check if a given 4-digit square number will yield another square number by adding 1 to each digit.

The program should first verify that the input is a four-digit square number. Then, 1 should be added to each digit. The newly formed number is checked to determine whether it is also a perfect square.

START

INPUT N

IF N is not a 4-digit number

    PRINT "Invalid input"

    STOP

END IF

IF N is not a perfect square

```
    PRINT "not a 4-digit square number"
    STOP
END IF
Extract the four digits:
d1, d2, d3, d4
IF any digit is 9
    PRINT "Cannot add 1 to every digit"
    STOP
END IF
newNumber ← (d1 + 1)*1000+ (d2 + 1)*100+ (d3 + 1) * 10+ (d4 + 1)
root ← integer square root of newNumber
IF root * root = newNumber
    PRINT " square number"
    PRINT newNumber
ELSE
    PRINT " not a square number"
    PRINT newNumber
END IF
END
```